U09 · Aplicaciones controladas por eventos

Hasta ahora tus programas tenían un guion claro: empezaban en `main`, ejecutaban sentencias en orden y terminaban. Con una interfaz gráfica eso cambia por completo: **ya no decides tú qué se ejecuta primero** — lo decide el usuario con cada clic, cada tecla, cada movimiento del ratón.

1. Programar “por eventos”

En una aplicación con interfaz gráfica (GUI) el programa se queda **esperando**. Cuando ocurre algo —el usuario pulsa un botón, escribe en un campo, cierra la ventana— el sistema genera un **evento**, y tu código solo tiene que decidir *qué hacer cuando ocurra*. Programar una GUI consiste, en buena parte, en escribir métodos que respondan a los eventos que te interesan.

Tres detalles que conviene tener claros desde el principio:

- **Todo es un objeto**: cada ventana, botón, color, tipo de letra… y cada evento. Una GUI en Java es programación orientada a objetos en estado puro.
- Java ha tenido tres librerías gráficas: **AWT** (la original, mínima), **Swing** (la que usamos en este curso, mucho más completa, construida sobre AWT) y **JavaFX** (la más moderna, con un diseño distinto). Los conceptos que aprendas aquí —eventos, listeners, layouts, MVC— se trasladan casi directamente a las otras.
- Las clases de Swing empiezan casi todas por `J` (`JFrame`, `JButton`, `JLabel`…), y viven en el paquete `javax.swing`. AWT tiene equivalentes sin la `J` (`Frame`, `Button`…) — no las confundas, no se mezclan bien.

2. Primeros pasos

La ventana más pequeña posible: un diálogo

```
import javax.swing.JOptionPane;

public class Hola {
    public static void main(String[] args) {
        JOptionPane.showMessageDialog(null, "¡Hola Mundo!");
    }
}
```

`null` significa “sin ventana padre”. Es un programa de consola que, de repente, muestra una ventanita — la forma más sencilla de usar Swing.

Tu propia ventana: `JFrame`

Una ventana de alto nivel (que no está dentro de otra) es un **frame**. En Swing, `JFrame`:

```
import javax.swing.*;

public class MiVentana extends JFrame {
    public MiVentana() {
        setTitle("Mi primera ventana");
        setSize(400, 300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);   // al cerrar la ventana,
termina el programa
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            new MiVentana().setVisible(true);   // un frame nace invisible: hay que
mostrarlo
        });
    }
}
```

Tres cosas importantes:

- Un `JFrame` recién creado mide **0 × 0 píxeles** y está **invisible** — por eso hay que darle tamaño (`setSize`) y mostrarlo (`setVisible(true)`). Esto te deja añadirle componentes antes de que el usuario lo vea.

- Sin `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)`, cerrar la ventana solo la oculta: el programa sigue ejecutándose en segundo plano, sin ventana.
- **`SwingUtilities.invokeLater(...)`** (o `EventQueue.invokeLater`) arranca la interfaz en el **hilo de despacho de eventos** (*Event Dispatch Thread*, EDT) — el hilo especial donde Swing procesa todos los eventos. Crear y modificar componentes Swing desde ese hilo es la regla de oro: hacerlo desde `main` suele funcionar… hasta que un día, de forma intermitente y casi imposible de reproducir, no funciona.

Contenedores y componentes

Un `JFrame` es un **contenedor**: contiene a otros componentes. Su zona útil (la que no incluye la barra de título ni los bordes, que dibuja el sistema operativo y no Swing) se llama **content pane**, y es donde añades tus componentes:

```
add(new JLabel("Escribe tu nombre:"));
add(new JButton("Aceptar"));
```

Un `JPanel` es otro contenedor, mucho más ligero que un frame — se usa para **agrupar** componentes dentro de la ventana y anidar unos contenedores dentro de otros.

3. Layouts: dónde va cada componente

Cuando el contenedor puede cambiar de tamaño, hay que decidir cómo se colocan sus componentes. De eso se encarga un **layout manager**, que se asigna con `setLayout(...)`:

Layout	Cómo los coloca	Se usa por defecto en
`FlowLayout`	En fila, de izquierda a derecha; salta de fila cuando no caben	`JPanel`
`BorderLayout`	Cinco zonas: `NORTH`, `SOUTH`, `EAST`, `WEST` y `CENTER` (que se queda con todo el espacio sobrante)	`JFrame`
`GridLayout`	Una rejilla de celdas del mismo tamaño (filas × columnas)	—

Layout	Cómo los coloca	Se usa por defecto en
`BoxLayout` / `GridBagLayout`	Más flexibles, para diseños complejos	—

```
setLayout(new BorderLayout());
add(new JLabel("Título"), BorderLayout.NORTH);
add(new JTextArea(), BorderLayout.CENTER);

JPanel botones = new JPanel(new GridLayout(1, 3, 5, 5));   // 1 fila, 3 columnas, huecos de 5 px
botones.add(new JButton("Uno"));
botones.add(new JButton("Dos"));
botones.add(new JButton("Tres"));
add(botones, BorderLayout.SOUTH);
```

⭐ **Be the Code**: casi cualquier interfaz real es una **combinación** de layouts: un `BorderLayout` general, con paneles dentro de sus zonas, y cada panel con su propio layout. Piensa la ventana como cajas dentro de cajas.

4. Eventos

El modelo de delegación

Java separa **quién genera** el evento de **quién lo atiende**:

- La **fuente** (*source*) es el componente donde ocurre algo (un `JButton` pulsado).
- El **listener** es un objeto que **escucha** esos eventos y contiene los métodos que se ejecutan cuando ocurren.
- Para que un listener reciba los eventos de un componente, hay que **registrarlo** en él: `componente.addXxxListener(listener)`.

Los pasos de siempre: importar `java.awt.event.*`, crear un objeto que implemente la interface *listener* adecuada, implementar sus métodos, y registrarlo en el componente.

El caso más habitual: pulsar un botón

Un clic en un botón genera un `ActionEvent`, que escucha un `ActionListener` — una interface **con un único método** (`actionPerformed`), así que puedes implementarla con una **lambda** (¡lo que aprendiste en la unidad 5 tiene uso real aquí!):

```
JTextField campoNombre = new JTextField(15);
JButton boton = new JButton("Saludar");

boton.addActionListener(e -> {
    String nombre = campoNombre.getText();
    JOptionPane.showMessageDialog(this, "Hola, " + nombre + "!");
});
```

Sin lambdas, tendrías que usar una clase anónima (`new ActionListener() { public void actionPerformed(ActionEvent e) { ... } }`) o una clase aparte — funcionalmente idéntico, mucho más verboso.

Eventos de ratón y teclado

Estos listeners tienen **varios** métodos (no son interfaces funcionales), así que no se pueden usar con lambda. Para no tener que implementar los cinco métodos de `MouseListener` cuando solo te interesa uno, Java incluye clases **adaptadoras** con todos los métodos ya vacíos — heredas de ella y sobrescribes solo el que necesitas:

```
panel.addMouseListener(new MouseAdapter() {
    @Override
    public void mousePressed(MouseEvent e) {
        System.out.println("Clic en (" + e.getX() + ", " + e.getY() + ")");
    }
});

campoTexto.addKeyListener(new KeyAdapter() {
    @Override
    public void keyPressed(KeyEvent e) {
        if (e.getKeyCode() == KeyEvent.VK_ENTER) {
            System.out.println("Has pulsado Intro");
        }
    }
});
```

El propio evento (`e`) te dice quién lo originó (`e.getSource()`), dónde ocurrió, qué tecla o botón se usó… — por eso **un mismo listener puede reutilizarse en varios componentes**.

Eventos de tiempo: `javax.swing.Timer`

Para ejecutar algo cada cierto tiempo (una animación, un reloj), Swing tiene su propio `Timer`, que lanza un `ActionEvent` periódicamente **dentro del hilo de eventos**:

```
Timer reloj = new Timer(1000, e ->
etiqueta.setText(LocalTime.now().toString()));
reloj.start();   // actualiza la etiqueta cada segundo
```

> ⚠ **Nunca bloquees el hilo de eventos**
>
> Todo el código de tus listeners se ejecuta en el hilo de despacho de eventos — el mismo que redibuja la ventana y atiende los clics. Si dentro de un listener haces algo lento (leer un fichero grande, una consulta a base de datos, un `Thread.sleep(5000)`), **la ventana se queda congelada** hasta que termine: no repinta, no responde a nada. Las tareas largas se hacen en otro hilo (lo estudiarás más adelante), nunca directamente en un listener.

5. Algunos componentes de la GUI

Componente	Para qué sirve	Se lee/escribe con
`JLabel`	Texto o imagen (no editable)	`setText`, `getText`
`JButton`	Botón	`addActionListener`
`JTextField` / `JPasswordField`	Una línea de texto editable	`getText`, `setText`
`JTextArea`	Varias líneas de texto (suele envolverse en un `JScrollPane`)	`getText`, `append`

Componente	Para qué sirve	Se lee/escribe con
`JCheckBox` / `JRadioButton`	Casilla / opción excluyente (los radio se agrupan en un `ButtonGroup`)	`isSelected`
`JComboBox`	Lista desplegable	`getSelectedItem`
`JList`	Lista visible de elementos	`getSelectedValue`
`JSlider` / `JSpinner`	Elegir un valor en un rango	`getValue`
`JTable`	Datos en filas y columnas	a través de un modelo

Y para los **cuadros de diálogo** ya hechos, `JOptionPane` ofrece cuatro variantes muy útiles:

```
JOptionPane.showMessageDialog(this, "Operación completada");                //
solo informar
int respuesta = JOptionPane.showConfirmDialog(this, "¿Seguro?");            // Sí /
No / Cancelar
String texto = JOptionPane.showInputDialog(this, "Tu nombre:");             // pedir
un texto
```

También existen los menús (`JMenuBar` → `JMenu` → `JMenuItem`), las barras de herramientas (`JToolBar`) y el selector de ficheros (`JFileChooser`, que conecta con lo que viste en la unidad 8).

6. El patrón MVC: separar la lógica de la pantalla

Cuando una aplicación crece, mezclar en la misma clase los botones, las operaciones y los datos se vuelve inmanejable. **MVC (Modelo-Vista-Controlador)** divide la aplicación en tres capas:

- **Modelo**: los datos y la lógica de negocio (sin saber nada de ventanas).

- **Vista**: la interfaz — los componentes que se ven y con los que interactúa el usuario (sin lógica de negocio).
- **Controlador**: el intermediario — escucha los eventos de la vista, llama al modelo y actualiza la vista con el resultado.

Ejemplo mínimo con una calculadora:

```
// MODELO: solo lógica, ni un JButton a la vista
public class CalculadoraModelo {
    public int sumar(int a, int b) { return a + b; }
    public int dividir(int a, int b) {
        if (b == 0) throw new IllegalArgumentException("No se puede dividir por
cero");
        return a / b;
    }
}

// CONTROLADOR: conecta vista y modelo
boton.addActionListener(e -> {
    try {
        int a = Integer.parseInt(campoA.getText());
        int b = Integer.parseInt(campoB.getText());
        resultado.setText(String.valueOf(modelo.sumar(a, b)));
    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(vista, "Introduce dos números enteros");
    }
});
```

La ventaja es enorme: puedes probar `CalculadoraModelo` sin abrir una sola ventana, o cambiar toda la interfaz (de Swing a JavaFX, por ejemplo) sin tocar la lógica de las operaciones.

7. Dibujar, imágenes y recursos

Dibujar sobre un componente

Para dibujar a mano (figuras, gráficos, animaciones), creas tu propio componente heredando de `JPanel` y sobrescribes **`paintComponent`**:

```
class Lienzo extends JPanel {
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);            // limpia el fondo: no te lo saltes
        g.setColor(Color.BLUE);
        g.fillOval(50, 50, 100, 100);
        g.setColor(Color.BLACK);
        g.drawString("Un círculo", 60, 180);
    }
}
```

Nunca llames a `paintComponent` tú mismo: Swing lo llama cuando hace falta (al mostrar la ventana, al redimensionarla…). Si quieres forzar un redibujado, llama a **`repaint()`**.

Imágenes y recursos

Una imagen sencilla en un componente se muestra con `ImageIcon`:

```
JLabel foto = new JLabel(new ImageIcon("foto.png"));
```

Pero esa ruta relativa depende de **desde qué carpeta se ejecuta el programa** — se rompe al empaquetar la aplicación en un `.jar`. Para imágenes y otros recursos que forman parte de tu proyecto, usa el *classpath*: coloca el fichero en una carpeta de recursos (en IntelliJ, una carpeta marcada como *Resources Root*) y cárgalo así:

```
URL url = getClass().getResource("/imagenes/logo.png");
JLabel logo = new JLabel(new ImageIcon(url));
```

Funciona igual ejecutándolo desde el IDE que desde el `.jar` final.

8. Diseñar interfaces desde IntelliJ IDEA

Escribir a mano todos los `add`, `setLayout` y `setBounds` es tedioso. IntelliJ IDEA incluye un **diseñador visual de interfaces Swing** (*Swing UI Designer*) — arrastras los componentes a una ventana y el IDE se encarga del código.

Crear un formulario: botón derecho sobre un paquete → **New → Swing UI Designer → GUI Form** (si no aparece, comprueba que el plugin *Swing UI Designer* está activado en *Settings → Plugins*). El IDE te crea dos ficheros enlazados: una clase Java y un fichero `.form` (la descripción visual).

El editor del diseñador tiene, en esencia:

- **Paleta** de componentes Swing, de donde los arrastras al formulario.
- **Árbol de componentes** (*Component Tree*), que muestra qué contiene a qué.
- **Panel de propiedades**, para cambiar el texto, el nombre del campo, el tamaño, el layout… de lo que tengas seleccionado.

Un truco imprescindible: dale a cada componente un **nombre de campo** significativo (*field name*) en el panel de propiedades (`botonAceptar`, `campoNombre`…) y marca *Custom Create* / *Bind to field* — así podrás referirte a él desde tu código Java. Para añadir el comportamiento, enlaza los listeners a mano en el constructor de la clase generada, igual que has visto en el apartado de eventos:

```
botonAceptar.addActionListener(e -> {
    // lo que debe ocurrir al pulsar el botón
});
```

> **Si tu proyecto usa módulos**
>
> Si el proyecto tiene un fichero `module-info.java`, añade `requires java.desktop;` para poder usar AWT y Swing; sin esa línea, el compilador no encontrará ni `JFrame` ni `JButton`.

RA y CE que cubre esta unidad

RA	Criterios de evaluación cubiertos
RA5. Realiza operaciones de entrada y salida de información…	f) herramientas del entorno de desarrollo para crear interfaces gráficos de usuario simples · g) controladores de eventos · h) programas con interfaces gráficos para la entrada y salida de información

Boletín inicial | Boletín intermedio | Extras